

Tipos Abstratos de Dados (TADs)

Principais implementações e complexidades

Tipos Abstratos de Dados vs. Estruturas de Dados

Conceitos Fundamentais

- **Tipo Abstrato de Dados (TAD):** Especificação matemática de um conjunto de dados e do conjunto de **operações** permitidas sobre eles, independente de implementação (o que a estrutura faz).
- **Estrutura de Dados: Forma concreta** de organizar e armazenar os dados na memória (como a estrutura faz).

Por que analisar implementações em PAA?

- Um mesmo TAD admite **múltiplas implementações** distintas.
- Cada implementação define os **tempos de execução (complexidades)** das operações.
- A escolha correta da estrutura de dados é frequentemente o fator determinante para a eficiência assintótica do algoritmo projetado.

TAD Lista: Conceito e Operações

Uma **lista** armazena uma sequência finita de elementos indexados por sua posição (0 até $n - 1$, onde n é o total de elementos).

Principais Operações

- **Acesso e Modificação por Posição:** Consultar ou alterar o elemento no índice i .
- **Inserção e Remoção nos Extremos:** Inserir/remover no início ou no final.
- **Inserção e Remoção Arbitrária:** Inserir ou remover elemento no índice i , ou remover por valor.
- **Busca e Verificação:** Verificar pertinência (`in`), encontrar índice (`index`), extremos (`min`, `max`) e ordenação (`sort`).
- **Subsequências e Estruturais:** Fatiamento (`slice`), concatenação (+) e repetição ($k \cdot L$).

TAD Lista: Implementação com Array Dinâmico

Implementação padrão de listas em Python (`list`). Os elementos ocupam posições contíguas de memória.

Operação	Sintaxe Python	Complexidade	Justificativa
Criação vazia	<code>l = []</code>	$O(1)$	Alocação de cabeçalho
Acessa posição	<code>v = l[i]</code>	$O(1)$	Cálculo direto de endereço
Altera posição	<code>l[i] = v</code>	$O(1)$	Cálculo direto de endereço
Insere no final	<code>l.append(v)</code>	$O(1)$ amortizado	Capacidade geométrica do array
Remove do final	<code>v = l.pop()</code>	$O(1)$	Apenas decrementa tamanho
Insere na posição	<code>l.insert(i, v)</code>	$O(n)$	Deslocamento de $n - i$ elementos
Remove da posição	<code>v = l.pop(i)</code>	$O(n)$	Deslocamento de $n - i - 1$ elementos
Remove valor	<code>l.remove(v)</code>	$O(n)$	Busca linear + deslocamento
Busca / Pertinência	<code>v in l</code>	$O(n)$	Varredura sequencial linear
Tamanho	<code>len(l)</code>	$O(1)$	Contador mantido em memória
Ordenação	<code>l.sort()</code>	$O(n \log n)$	Algoritmo de ordenação
Fatia (<i>slice</i>)	<code>l[a:b]</code>	$O(b - a)$	Cópia de bloco de elementos

Estrutura Auxiliar: Tuplas

Definição

Tuplas são sequências **imutáveis**: tamanho e referências dos elementos não podem ser modificados após a criação.

- **Sintaxe:** Utiliza parênteses: $t = (v1, v2, \dots, vn)$.
- **Operações suportadas:** Todas as operações de listas que **não realizam mutação** ($t[i]$, $\text{len}(t)$, $v \text{ in } t$, fatiamento, etc.).
- **Aplicações em Algoritmos:**
 - Retorno de múltiplos valores em funções.
 - Chaves compostas em dicionários e conjuntos (objetos imutáveis são **hasháveis**).
 - Elementos ordenáveis por **ordem lexicográfica** (essencial para Filas de Prioridade e Heaps).

TADs Pilha, Fila e Deque: Conceitos

Estruturas lineares com restrições nos pontos de inserção e remoção:

Pilha (LIFO)

Last-In, First-Out

- Inserção no topo: push
- Remoção no topo: pop
- Consulta topo: top

Fila (FIFO)

First-In, First-Out

- Inserção no fim: enqueue
- Remoção no início: dequeue
- Consulta início: front

Deque

Double-Ended Queue

- Insere início / fim
- Remove início / fim
- Acesso nos dois extremos

Problema da Implementação Ingênua com Array

- Para **Pilha**, array dinâmico é ideal: append e pop executam em tempo $O(1)$.
- Para **Fila**, remover no início de um array (`l.pop(0)`) custa $O(n)$, tornando a fila ineficiente!

Deque com Lista Duplamente Encadeada

Implementação fornecida pelo módulo `collections.deque`. Mantém ponteiros para o primeiro e o último nó.

Operação	Sintaxe Python	Deque (Encadeada)	Lista (Array)
Criação vazia	<code>q = deque([])</code>	$O(1)$	$O(1)$
Inserir no início	<code>q.appendleft(v)</code>	$O(1)$	$O(n)$
Inserir no final	<code>q.append(v)</code>	$O(1)$	$O(1)$ amort.
Remover do início	<code>v = q.popleft()</code>	$O(1)$	$O(n)$
Remover do final	<code>v = q.pop()</code>	$O(1)$	$O(1)$
Acessa posição arbitrária	<code>v = q[i]</code>	$O(n)$	$O(1)$
Altera posição arbitrária	<code>q[i] = v</code>	$O(n)$	$O(1)$
Tamanho	<code>len(q)</code>	$O(1)$	$O(1)$

Compromisso de Projeto (*Trade-off*)

- Ganha-se $O(1)$ para modificações em ambas as extremidades (fila eficiente).
- Perde-se o acesso em tempo constante a posições no meio da coleção ($O(n)$).

TAD Dicionário (Mapa): Conceito e Operações

Um **dicionário** associa chaves exclusivas a valores correspondentes (conjunto de pares (k, v)).

Características

- Enquanto listas indexam apenas por inteiros contíguos de 0 a $n - 1$, o dicionário aceita qualquer objeto **imutável** (número, string, tupla) como chave.
- Não permite chaves duplicadas (cada chave aponta para um único valor).

Principais Operações

- **Inserção e Alteração:** Associar chave k ao valor v .
- **Acesso:** Recuperar valor associado à chave k .
- **Remoção:** Eliminar a chave k e seu respectivo valor.
- **Pertinência:** Testar se uma chave k existe no mapeamento.
- **Enumeração:** Obter listas de chaves, valores ou pares (k, v) .

TAD Dicionário: Implementação com Tabela Hash

Implementação padrão de dicionários em Python (`dict`). As complexidades referem-se ao **caso médio**.

Operação	Sintaxe Python	Complexidade	Observação
Criação vazia	<code>d = {}</code>	$O(1)$	Alocação da tabela
Insere / Altera	<code>d[k] = v</code>	$O(1)$	Média (calcula hash e insere)
Acessa por chave	<code>v = d[k]</code>	$O(1)$	Exceção se $k \notin d$
Acessa com padrão	<code>v = d.get(k, u)</code>	$O(1)$	Retorna u se $k \notin d$
Remove chave	<code>v = d.pop(k)</code>	$O(1)$	Remove e retorna v
Pertinência da chave	<code>k in d</code>	$O(1)$	Verificação direta via hash
Menor / Maior chave	<code>min(d), max(d)</code>	$O(n)$	Tabela não é ordenada
Tamanho	<code>len(d)</code>	$O(1)$	Mantém contador de chaves
Iteração nas chaves	<code>for k in d</code>	$O(n)$	Percorre todas as entradas
Lista de chaves	<code>list(d.keys())</code>	$O(n)$	Cópia das chaves
Lista de pares	<code>list(d.items())</code>	$O(n)$	Lista de tuplas (k, v)

- **Pior Caso Teórico:** $O(n)$ se houver colisão de todas as chaves no mesmo *bucket* (improvável com boas funções de dispersão).

TAD Conjunto: Conceito e Implementação

O **conjunto** modela a entidade matemática de mesmo nome: coleção de elementos **não repetidos** e **sem ordem intrínseca**.

Implementação com Tabela Hash (set)

Implementado como uma tabela hash onde apenas as chaves são armazenadas (valores são omitidos).

Operação	Sintaxe Python	Complexidade	Descrição
Cria conjunto vazio	<code>A = set()</code>	$O(1)$	Alocação da tabela
Insere elemento	<code>A.add(v)</code>	$O(1)$	Insere se não presente
Remove elemento	<code>A.remove(v)</code>	$O(1)$	Lança erro se $v \notin A$
Pertinência	<code>v in A</code>	$O(1)$	Verificação direta via hash
Tamanho	<code>len(A)</code>	$O(1)$	Total de elementos únicos
União	<code>C = A B</code>	$O(A + B)$	Constrói novo conjunto
Interseção	<code>C = A & B</code>	$O(\min(A , B))$	Itera no menor, busca no maior
Diferença	<code>C = A - B</code>	$O(A)$	Itera em A, filtra os de B
Subconjunto ($A \subseteq B$)	<code>A <= B</code>	$O(A)$	Testa pertinência de cada $v \in A$
Igualdade ($A == B$)	<code>A == B</code>	$O(A)$	$O(1)$ se tamanhos forem distintos

TAD Multiconjunto (*Bag*): Conceito e Operações

Um **multiconjunto** é semelhante a um conjunto (não possui ordem), porém **permite elementos repetidos**.

Implementação Eficiente com Tabela Hash (`multiset.Multiset`)

Mapeia cada elemento (chave) ao seu respectivo contador de ocorrências (inteiro).

Operação	Sintaxe Python	Complexidade	Descrição
Cria multiconjunto vazio	<code>A = multiset()</code>	$O(1)$	Alocação da tabela
Insere elemento	<code>A.add(v)</code>	$O(1)$	Incrementa multiplicidade de v
Remove elemento	<code>A.remove(v)</code>	$O(1)$	Decrementa multiplicidade de v
Define # cópias	<code>A[v] = c</code>	$O(1)$	Atualiza multiplicidade de v
Acessa # cópias	<code>c = A[v]</code>	$O(1)$	Retorna contagem (0 se $v \notin A$)
Pertinência	<code>v in A</code>	$O(1)$	Verdadeiro se multiplicidade > 0
Tamanho total	<code>len(A)</code>	$O(1)$	Soma das multiplicidades
União multiconjunto	<code>C = A B</code>	$O(A + B)$	Multiplicidade: $\max(A[v], B[v])$
Interseção multiconjunto	<code>C = A & B</code>	$O(\min(A , B))$	Multiplicidade: $\min(A[v], B[v])$
Diferença multiconjunto	<code>C = A - B</code>	$O(A)$	Multiplicidade: $\max(0, A[v] - B[v])$
Sub-multiconjunto	<code>A <= B</code>	$O(A)$	Verifica se $\forall v : A[v] \leq B[v]$

TAD Conjuntos Disjuntos: Union-Find

Mantém partição de um universo de n elementos em subconjuntos **disjuntos** (s/ interseção).

Operações Centrais

- `find(e)`: Retorna o representante único (identificador) do conjunto que contém e .
- `union(e1, e2)`: Une os conjuntos que contêm e_1 e e_2 .
- `connected(e1, e2)`: Verifica se `find(e1) == find(e2)`.

Operação	Sintaxe Python	Complexidade	Detalhe da Implementação
Cria inicializado	<code>U = UnionFind([e1,...,en])</code>	$O(n)$	Cada elemento em seu próprio conjunto
Inserir elemento	<code>U.add(e)</code>	$O(1)$	Cria novo conjunto unitário
Busca representante	<code>U.find(e)</code>	$O(1)^*$	Praticamente constante ($O(\alpha(n))$)
União	<code>U.union(e1, e2)</code>	$O(1)^*$	Praticamente constante ($O(\alpha(n))$)
Conectados	<code>U.connected(e1, e2)</code>	$O(1)^*$	Dois chamadas a <code>find</code>
Componente de e	<code>U.component(e)</code>	$O(n)$	Retorna elementos do grupo
Todas componentes	<code>U.components()</code>	$O(n)$	m conjuntos e n elementos

- (*) Implementado com **união por rank/peso** e **compressão de caminhos**; $\alpha(n) \leq 4$ para qualquer entrada realista.

TAD Conjunto Ordenado: Motivação

Cenário: o algoritmo precisa realizar frequentes **inserções**, **remoções** e simultaneamente consultas ao ***i*-ésimo menor elemento** (ou mínimo/máximo).

Comparativo de Custo Total (para n inserções/remoções e n buscas)

① Array não ordenado:

- Inserção: $O(1)$ — Remoção: $O(n)$ — i -ésimo menor: $O(n)$
- Custo total após n operações: $O(n^2)$

② Array ordenado:

- Inserção e Remoção: $O(n)$ (deslocamento) — i -ésimo menor: $O(1)$ (índice direto)
- Custo total após n operações: $O(n^2)$

③ Árvore Binária Balanceada (AVL, Rubro-Negra):

- Inserção, Remoção e Busca por ordem estatística (i -ésimo): todas em $O(\log n)$
- Custo total após n operações: $O(n \log n)$

TAD Conjunto Ordenado: Complexidades

Operações baseadas em **árvore balanceada** com indexação posicional (classe SortedSet).

Operação	Sintaxe Python	Complexidade	Descrição
Criação vazia	<code>S = SortedSet()</code>	$O(1)$	Inicializa estrutura
Cria inicializado	<code>S = SortedSet(L)</code>	$O(n \log n)$	Insere e balanceia n nós
Insere elemento	<code>S.add(v)</code>	$O(\log n)$	Busca folha e rebalanceia
Remove elemento	<code>S.remove(v)</code>	$O(\log n)$	Remove nó e rebalanceia
i -ésimo menor	<code>S[i]</code>	$O(\log n)$	Consulta usando subárvores contadas
Mínimo elemento	<code>v = S[0]</code>	$O(\log n)$	Acesso ao nó mais à esquerda
Máximo elemento	<code>v = S[-1]</code>	$O(\log n)$	Acesso ao nó mais à direita
Pertinência	<code>v in S</code>	$O(1)$ ou $O(\log n)$	$O(1)$ com hash auxiliar
Tamanho	<code>len(S)</code>	$O(1)$	Contador interno da raiz

Observação Prática

Bibliotecas como `sortedcontainers` utilizam internamente **Árvores-B**, com complexidade amortizada teórica $O(\sqrt[3]{n})$, mas com constantes menores.

TAD Fila de Prioridade: Heap Binária

TAD focado exclusivamente em **inserção** e recuperação/remoção do elemento de **extrema prioridade** (menor ou maior elemento).

Implementação com Heap Binária (heapq)

A heap é mantida diretamente dentro de um array convencional (`list`) sem ponteiros adicionais: para o nó no índice i , seus filhos residem em $2i + 1$ e $2i + 2$.

Operação	Sintaxe Python	Complexidade	Observação
Cria vazia	<code>h = []</code>	$O(1)$	Lista comum
Cria inicializada	<code>heapify(h)</code>	$O(n)$	Algoritmo de Floyd (<i>sift-down</i>)
Insere elemento	<code>heappush(h, v)</code>	$O(\log n)$	Borbulha para o topo (<i>sift-up</i>)
Remove menor	<code>v = heappop(h)</code>	$O(\log n)$	Extraí raiz e recompõe heap
Consulta menor	<code>v = h[0]</code>	$O(1)$	Menor elemento na raiz do array
Altera prioridade	<code>increase/decrease_key</code>	$O(\log n)$	Atualiza e reposiciona elemento

- Para heap de **máximo**: armazena-se os números com sinal invertido (ou chaves lexicográficas invertidas em tuplas (k, v)).

Fila de Prioridade vs. Conjunto Ordenado

Operação	Heap Binária	Conjunto Ordenado (Árvore)
Criação inicializada	$O(n)$	$O(n \log n)$
Consulta ao mínimo	$O(1)$	$O(\log n)$
Remove menor	$O(\log n)$	$O(\log n)$
Insere elemento	$O(\log n)$	$O(\log n)$
Remove elemento arbitrário	Ineficiente ($O(n)$)	$O(\log n)$
Acessa i -ésimo menor	Não suportado	$O(\log n)$

Regra de Decisão de Projeto

- Se o algoritmo requer **apenas** inserções e extração de extremos (como nos algoritmos de Dijkstra e Prim), a **Heap** é superior: $O(1)$ no mínimo, construção inicial linear em $O(n)$ e constantes multiplicativas muito menores.
- Se for necessário consultar elementos por posição relativa (i -ésimo) ou remover elementos arbitrários, o **Conjunto Ordenado** é obrigatório.

TAD Grafo: Conceito e Representações

Um **grafo** $G = (V, E)$ é composto por um conjunto de vértices V ($|V| = n$) e um conjunto de arestas E ($|E| = m$).

Matriz de Adjacência

- Espaço em memória: $O(n^2)$.
- Testa se $(u, v) \in E$: $O(1)$.
- Itera nos vizinhos de u : $O(n)$.
- Ineficiente para grafos esparsos ($m \ll n^2$).

Lista de Adjacência

- Espaço em memória: $O(n + m)$.
- Itera nos vizinhos de u : $O(d_u)$, onde d_u é o grau do vértice u .
- Somatório dos graus: $\sum_u d_u \in O(m)$.
- Ideal para algoritmos de busca (BFS, DFS, Dijkstra, Prim).

TAD Grafo com Dicionários de Adjacências

Representação padrão: dicionário de dicionários (para grafos com pesos) ou dicionário de conjuntos. Seja $G[u][v] = w$ o peso da aresta $\{u, v\}$.

Operação	Sintaxe Python	Complexidade	Descrição
Cria grafo vazio	<code>G = {}</code>	$O(1)$	Inicializa mapeamento
Inserir vértice	<code>G[u] = {}</code>	$O(1)$	Adiciona chave com adjacências vazias
Inserir aresta (u, v)	<code>G[u][v] = w</code>	$O(1)$	Inserir chave v no dict $G[u]$
Remover aresta (u, v)	<code>G[u].pop(v)</code>	$O(1)$	Remover v do dicionário $G[u]$
Remover vértice u	<code>G.pop(u)</code>	$O(1)$	Exclui u e suas arestas de saída
Verifica adjacência	<code>v in G[u]</code>	$O(1)$	Consulta hash direta no conjunto/dict
Itera nos vértices	<code>for u in G:</code>	$O(n)$	Percorre as n chaves de G
Itera nos vizinhos	<code>for v in G[u]:</code>	$O(d_u)$	Proporcional ao grau do vértice u

Vantagem em Análise Assintótica

Varredura completa sobre todos os adjacentes de todos os vértices:

$$\sum_{u \in V} O(d_u) = O(m)$$

Quadro Resumo: Complexidades por TAD e Implementação

TAD	Implementação Adotada	Acesso Pos.	Insere Fim	Insere Início	Busca / Contido	Mínimo
Lista	Array Dinâmico (<i>list</i>)	$O(1)$	$O(1)$ amort.	$O(n)$	$O(n)$	$O(n)$
Deque	Lista Duplamente Encadeada	$O(n)$	$O(1)$	$O(1)$	$O(n)$	$O(n)$
Dicionário	Tabela Hash (<i>dict</i>)	–	$O(1)$ (chave)	–	$O(1)$ (chave)	$O(n)$
Conjunto	Tabela Hash (<i>set</i>)	–	$O(1)$	–	$O(1)$	$O(n)$
Conj. Disjunto	Union-Find (<i>rank + comp.</i>)	–	$O(1)$ (add)	–	$O(1)^*$ (find)	–
Conj. Ordenado	Árvore Balanceada	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(1)$ / $O(\log n)$	$O(\log n)$
Fila Prioridade	Heap Binária (<i>heapq</i>)	–	$O(\log n)$	–	$O(n)$	$O(1)$
Grafo	Dict de Adjacências	–	$O(1)$ (edge)	–	$O(1)$ (adj)	–

Conclusão

- Não existe estrutura ótima para todos os cenários.
- A formulação da complexidade de um algoritmo depende estritamente das operações dominantes requeridas e da implementação eleita para cada TAD envolvido.